

Line-by-Line Mathematical Guide for HAETAE_Distributions.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Sampling distributions, support facts, and point bounds.

How To Read This Script

Read this file as the probability-law library for the proof. Its scripts justify losslessness, support membership, and sampler bounds.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore Distr DInterval DList IntDiv List Real StdOrder.</code>	Imports AllCore Distr DInterval DList IntDiv List Real StdOrder. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_Algebra.</code>	Imports HAETAE_Params HAETAE_Algebra. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_Distributions.</code>	Begins the namespace HAETAE_Distributions, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Algebra.</code>	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
7	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
8	<code>import RealOrder.</code>	Opens RealOrder so its names can be used without qualification in the following proof.
9	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
10	<code>type signing_randomness_source = int.</code>	Declares carrier set signing_randomness_source. EasyCrypt will type-check all later functions and games over this mathematical domain.
11	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
12	<code>op dbyte : byte distr = dinter 0 255.</code>	Defines dbyte as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
13	<code>op dseed : seed distr = dlist dbyte seedbytes.</code>	Defines dseed as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.

Line	EasyCrypt source	Mathematical reading
14	op dcrh : crh distr = dlist dbyte crhbytes.	Defines dcrh as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
15	op signing_entropy_token_cardinality : int = 288230376151711744.	Defines integer-valued parameter signing_entropy_token_cardinality, used for dimensions, budgets, support sizes, or bounds.
16	op signing_entropy_token_in_range (x : int) : bool =	Defines predicate signing_entropy_token_in_range, an event or well-formedness condition used in later implications and game proofs.
17	0 <= x /\ x < signing_entropy_token_cardinality.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
18	op signing_randomness_byte_ok (b : byte) : bool = 0 <= b /\ b < 256.	Defines predicate signing_randomness_byte_ok, an event or well-formedness condition used in later implications and game proofs.
19	op signing_entropy_token_to_coins (x : int) : random_coins =	Defines integer-valued parameter signing_entropy_token_to_coins, used for dimensions, budgets, support sizes, or bounds.
20	[x %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
21	(x %/ 256) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
22	(x %/ 65536) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
23	(x %/ 16777216) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
24	(x %/ 4294967296) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
25	(x %/ 1099511627776) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	(x %/ 281474976710656) %% 256;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	x %/ 72057594037927936] ++	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
28	nseq (seedbytes - 8) 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
29	op signing_randomness_domain (coins : random_coins) : bool =	Defines predicate signing_randomness_domain, an event or well-formedness condition used in later implications and game proofs.
30	size coins = seedbytes /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
31	all signing_randomness_byte_ok coins /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
32	signing_entropy_token_in_range (signing_entropy_token_of_coins coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
33	op dsigning_entropy_token : int distr =	Defines dsigning_entropy_token as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
34	dinter 0 (signing_entropy_token_cardinality - 1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
35	op dsigning_randomness_source : signing_randomness_source distr =	Defines dsigning_randomness_source as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
36	dsigning_entropy_token.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
37	op signing_randomness_source_valid (src : signing_randomness_source) : bool =	Defines predicate signing_randomness_source_valid, an event or well-formedness condition used in later implications and game proofs.
38	signing_entropy_token_in_range src.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
39	op signing_randomness_from_source (src : signing_randomness_source) :	Defines operation signing_randomness_from_source, a total mathematical function or abbreviation used by later declarations.
40	random_coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
41	signing_entropy_token_to_coins src.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	op dsigning_randomness : random_coins distr =	Defines dsigning_randomness as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
43	dmap dsigning_randomness_source signing_randomness_from_source.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
44	op drandom_coins : random_coins distr = dsigning_randomness.	Defines drandom_coins as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
45	op signing_randomness_point_bound : real =	Defines real-valued term signing_randomness_point_bound, usually a loss, probability bound, or advantage expression.
46	1%r / (signing_entropy_token_cardinality)%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
47	op signing_randomness_token_spread (d : random_coins distr) : bool =	Defines signing_randomness_token_spread as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
48	forall token,	Part of a universal mathematical condition that must hold for all listed values.
49	mu d (fun coins => signing_entropy_token_of_coins coins = token) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
50	signing_randomness_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
51	op signing_randomness_distribution_ok (d : random_coins distr) : bool =	Defines signing_randomness_distribution_ok as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
52	is_lossless d /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	(forall coins, coins \in d => signing_randomness_domain coins) /\	Part of a universal mathematical condition that must hold for all listed values.
54	signing_randomness_token_spread d.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
55	op dcoeff : coeff distr = dinter 0 (q - 1).	Defines dcoeff as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
56	op dpoly : poly distr = dlist dcoeff n.	Defines dpoly as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
57	op dpolyveck (md : mode) : polyveck distr = dlist dpoly (mode_k md).	Defines dpolyveck as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
58	op dpolyvecl (md : mode) : polyvecl distr = dlist dpoly (mode_l md).	Defines dpolyvecl as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
59	op dmatrix (md : mode) : matrix distr = dlist (dpolyvecl md) (mode_k md).	Defines dmatrix as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
60	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
61	(* This is the structural prefix-sparse challenge carrier distribution over a	Commentary for the human reader; it does not add a checked mathematical assumption.
62	byte-list seed. It is checked against the local challenge-shape predicate,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
63	but still needs a paper-faithful sampler/correspondence proof before claiming	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
64	equality with the HAETAE specification distribution. *)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	op dchallenge (md : mode) : challenge distr =	Defines dchallenge as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
66	dmap dcrh (challenge_from_seed md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
67	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
68	type signing_sample_pair = polyvecl * polyveck.	Declares carrier set signing_sample_pair. EasyCrypt will type-check all later functions and games over this mathematical domain.
69	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
70	op signing_sample_pair_y1 (s : signing_sample_pair) : polyvecl = s.'1.	Defines operation signing_sample_pair_y1, a total mathematical function or abbreviation used by later declarations.
71	op signing_sample_pair_y2 (s : signing_sample_pair) : polyveck = s.'2.	Defines operation signing_sample_pair_y2, a total mathematical function or abbreviation used by later declarations.
72	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
73	op signing_sample_pair_wf (md : mode) (s : signing_sample_pair) : bool =	Defines predicate signing_sample_pair_wf, an event or well-formedness condition used in later implications and game proofs.
74	polyvecl_wf md (signing_sample_pair_y1 s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
75	polyveck_wf md (signing_sample_pair_y2 s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
76	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
77	op signing_sample_pair_unit (md : mode) (s : signing_sample_pair) : bool =	Defines predicate signing_sample_pair_unit, an event or well-formedness condition used in later implications and game proofs.
78	polyvecl_unit md (signing_sample_pair_y1 s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
79	polyveck_unit md (signing_sample_pair_y2 s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
80	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
81	op signing_sample_bound (md : mode) : int = mode_ln md.	Defines integer-valued parameter signing_sample_bound, used for dimensions, budgets, support sizes, or bounds.
82	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
83	op coeff_bounded_by (b : int) (x : coeff) : bool =	Defines predicate coeff_bounded_by, an event or well-formedness condition used in later implications and game proofs.
84	-b <= x /\ x <= b.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
85	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
86	op poly_bounded_by (b : int) (p : poly) : bool =	Defines predicate poly_bounded_by, an event or well-formedness condition used in later implications and game proofs.
87	poly_wf p /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
88	forall i, 0 <= i < n =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
89	coeff_bounded_by b (poly_coeff p i).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
90	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
91	op polyvecl_bounded_by (md : mode) (b : int) (xs : polyvecl) : bool =	Defines predicate polyvecl_bounded_by, an event or well-formedness condition used in later implications and game proofs.
92	polyvecl_wf md xs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
93	forall row, 0 <= row < mode_l md =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
94	poly_bounded_by b (nth poly_zero xs row).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
95	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
96	op polyveck_bounded_by (md : mode) (b : int) (xs : polyveck) : bool =	Defines predicate polyveck_bounded_by, an event or well-formedness condition used in later implications and game proofs.
97	polyveck_wf md xs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
98	forall row, 0 <= row < mode_k md =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
99	poly_bounded_by b (nth poly_zero xs row).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
100	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
101	op signing_sample_pair_bounded (md : mode) (s : signing_sample_pair) : bool =	Defines predicate signing_sample_pair_bounded, an event or well-formedness condition used in later implications and game proofs.
102	polyvecl_bounded_by md (signing_sample_bound md)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
103	(signing_sample_pair_y1 s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
104	polyveck_bounded_by md (signing_sample_bound md)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
105	(signing_sample_pair_y2 s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
106	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
107	op signing_sample_pair_sample_ok (md : mode) (s : signing_sample_pair) : bool =	Defines predicate signing_sample_pair_sample_ok, an event or well-formedness condition used in later implications and game proofs.
108	signing_sample_pair_wf md s /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
109	signing_sample_pair_bounded md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
110	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
111	op signing_sample_pair_of_coins	Defines operation signing_sample_pair_of_coins, a total mathematical function or abbreviation used by later declarations.
112	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
113	(coins : random_coins) : signing_sample_pair =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
114	(signing_sample_y1 md sk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
115	signing_sample_y2 md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
116	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
117	op dstructural_signing_sample_pair	Defines operation <code>dstructural_signing_sample_pair</code> , a total mathematical function or abbreviation used by later declarations.
118	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
119	(coins : random_coins) : signing_sample_pair distr =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
120	dunit (signing_sample_pair_of_coins md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
121	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
122	op dsigning_unit_coeff : coeff distr = duniform [-1; 1].	Defines <code>dsigning_unit_coeff</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
123	op dsigning_unit_poly : poly distr = dlist dsigning_unit_coeff n.	Defines <code>dsigning_unit_poly</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
124	op dsigning_unit_polyvecl (md : mode) : polyvecl distr =	Defines <code>dsigning_unit_polyvecl</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
125	dlist dsigning_unit_poly (mode_l md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
126	op dsigning_unit_polyveck (md : mode) : polyveck distr =	Defines <code>dsigning_unit_polyveck</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
127	dlist dsigning_unit_poly (mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
128	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
129	op signing_bounded_coeff_cardinality (md : mode) : int =	Defines integer-valued parameter <code>signing_bounded_coeff_cardinality</code> , used for dimensions, budgets, support sizes, or bounds.
130	2 * signing_sample_bound md + 1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
131	op signing_bounded_coeff_point_bound (md : mode) : real =	Defines real-valued term <code>signing_bounded_coeff_point_bound</code> , usually a loss, probability bound, or advantage expression.
132	1/r / (signing_bounded_coeff_cardinality md)%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
133	op dsigning_bounded_coeff (md : mode) : coeff distr =	Defines <code>dsigning_bounded_coeff</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
134	dinter (-(signing_sample_bound md)) (signing_sample_bound md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
135	op dsigning_bounded_poly (md : mode) : poly distr =	Defines <code>dsigning_bounded_poly</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
136	dlist (dsigning_bounded_coeff md) n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
137	op dsigning_bounded_polyvecl (md : mode) : polyvecl distr =	Defines <code>dsigning_bounded_polyvecl</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
138	dlist (dsigning_bounded_poly md) (mode_l md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
139	op dsigning_bounded_polyveck (md : mode) : polyveck distr =	Defines <code>dsigning_bounded_polyveck</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
140	dlist (dsigning_bounded_poly md) (mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
141	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
142	(* Entropy-bearing checked signing-sample source. This is the first	Commentary for the human reader; it does not add a checked mathematical assumption.
143	non-deterministic sampler instance for the attempt-state boundary; the	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
144	paper's hyperball sampler still has to replace this bounded-unit source. *)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
145	op dbounded_unit_signing_sample_pair (md : mode) :	Defines operation <code>dbounded_unit_signing_sample_pair</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
146	<code>signing_sample_pair distr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
147	<code>dlet (dsigning_unit_polyvecl md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
148	<code>(fun y1 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
149	<code>dmap (dsigning_unit_polyveck md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
150	<code>(fun y2 => (y1, y2))).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
151	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
152	<code>(* Paper-shaped bounded carrier for the future HAETAE hyperball sampler. This</code>	Commentary for the human reader; it does not add a checked mathematical assumption.
153	<code>checks dimensions, coefficient bounds, and point-bound facts for a product</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
154	<code>bounded source; it still has to be replaced by the exact hyperball/sphere</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
155	<code>sampler and rejection loop correspondence. *)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
156	<code>op dchecked_hyperball_signing_sample_pair (md : mode) :</code>	Defines operation <code>dchecked_hyperball_signing_sample_pair</code> , a total mathematical function or abbreviation used by later declarations.
157	<code>signing_sample_pair distr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
158	<code>dlet (dsigning_bounded_polyvecl md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
159	<code>(fun y1 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
160	<code>dmap (dsigning_bounded_polyveck md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
161	<code>(fun y2 => (y1, y2))).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
162	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
163	<code>op hyperball_coeff_ok (md : mode) (c : coeff) : bool =</code>	Defines predicate <code>hyperball_coeff_ok</code> , an event or well-formedness condition used in later implications and game proofs.
164	<code>coeff_bounded_by (signing_sample_bound md) c.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
165	<code>op hyperball_poly_ok (md : mode) (p : poly) : bool =</code>	Defines predicate <code>hyperball_poly_ok</code> , an event or well-formedness condition used in later implications and game proofs.
166	<code>poly_bounded_by (signing_sample_bound md) p.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
167	<code>op hyperball_polyvecl_ok (md : mode) (y1 : polyvecl) : bool =</code>	Defines predicate <code>hyperball_polyvecl_ok</code> , an event or well-formedness condition used in later implications and game proofs.
168	<code>polyvecl_bounded_by md (signing_sample_bound md) y1.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
169	<code>op hyperball_polyveck_ok (md : mode) (y2 : polyveck) : bool =</code>	Defines predicate <code>hyperball_polyveck_ok</code> , an event or well-formedness condition used in later implications and game proofs.
170	<code>polyveck_bounded_by md (signing_sample_bound md) y2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
171	<code>op hyperball_sample_ok (md : mode) (s : signing_sample_pair) : bool =</code>	Defines predicate <code>hyperball_sample_ok</code> , an event or well-formedness condition used in later implications and game proofs.
172	<code>hyperball_polyvecl_ok md (signing_sample_pair_y1 s) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
173	<code>hyperball_polyveck_ok md (signing_sample_pair_y2 s).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
174	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
175	<code>op dhyperball_coeff (md : mode) : coeff distr =</code>	Defines <code>dhyperball_coeff</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
176	<code>dsigning_bounded_coeff md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
177	<code>op dhyperball_poly (md : mode) : poly distr =</code>	Defines <code>dhyperball_poly</code> as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
178	<code>dsigning_bounded_poly md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
179	op dhyperball_polyvecl (md : mode) : polyvecl distr =	Defines dhyperball_polyvecl as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
180	dsigning_bounded_polyvecl md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
181	op dhyperball_polyveck (md : mode) : polyveck distr =	Defines dhyperball_polyveck as a distribution-valued object; later probability proofs rely on its losslessness, support, or point bounds.
182	dsigning_bounded_polyveck md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
183	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
184	op hyperball_coeff_point_bound (md : mode) : real =	Defines real-valued term hyperball_coeff_point_bound, usually a loss, probability bound, or advantage expression.
185	signing_bounded_coeff_point_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
186	op hyperball_poly_point_bound (md : mode) : real =	Defines real-valued term hyperball_poly_point_bound, usually a loss, probability bound, or advantage expression.
187	hyperball_coeff_point_bound md ^ n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
188	op hyperball_polyvecl_point_bound (md : mode) : real =	Defines real-valued term hyperball_polyvecl_point_bound, usually a loss, probability bound, or advantage expression.
189	hyperball_poly_point_bound md ^ (mode_l md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
190	op hyperball_polyveck_point_bound (md : mode) : real =	Defines real-valued term hyperball_polyveck_point_bound, usually a loss, probability bound, or advantage expression.
191	hyperball_poly_point_bound md ^ (mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
192	op hyperball_sample_pair_point_bound (md : mode) : real =	Defines real-valued term hyperball_sample_pair_point_bound, usually a loss, probability bound, or advantage expression.
193	hyperball_polyvecl_point_bound md *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
194	hyperball_polyveck_point_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
195	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
196	(* Exact sampler interface for the next HAETAE replacement step. The current	Commentary for the human reader; it does not add a checked mathematical assumption.
197	compiled implementation is the checked bounded carrier above, presented as a	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
198	product sampler so point-probability bounds can be exposed explicitly. The	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
199	paper-faithful sphere/hyperball law still has to replace the component	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
200	distributions behind this interface. *)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
201	op dexact_hyperball_signing_sample_pair (md : mode) :	Defines operation dexact_hyperball_signing_sample_pair, a total mathematical function or abbreviation used by later declarations.
202	signing_sample_pair distr =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
203	(dhyperball_polyvecl md) ‘*’ (dhyperball_polyveck md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
204	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
205	op signing_sample_pair_distribution_ok	Defines operation signing_sample_pair_distribution_ok, a total mathematical function or abbreviation used by later declarations.
206	(md : mode) (d : signing_sample_pair distr) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
207	is_lossless d /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
208	(forall s, s \in d =>	Part of a universal mathematical condition that must hold for all listed values.
209	signing_sample_pair_sample_ok md s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
210	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
211	lemma byte_distribution_lossless : is_lossless dbyte.	States checked lemma byte_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
212	<code>proof. by rewrite /dbyte; apply dinter_ll. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
213	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
214	<code>lemma seed_distribution_lossless : is_lossless dseed.</code>	States checked lemma <code>seed_distribution_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
215	<code>proof.</code>	Begins the proof script for the preceding proposition.
216	<code>rewrite /dseed.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
217	<code>by apply dlist_ll; apply byte_distribution_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
218	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
219	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
220	<code>lemma crh_distribution_lossless : is_lossless dcrh.</code>	States checked lemma <code>crh_distribution_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
221	<code>proof.</code>	Begins the proof script for the preceding proposition.
222	<code>rewrite /dcrh.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
223	<code>by apply dlist_ll; apply byte_distribution_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
224	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
225	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
226	<code>lemma signing_entropy_token_cardinality_positive :</code>	States checked lemma <code>signing_entropy_token_cardinality_positive</code> . The following proof script must derive this proposition from prior definitions and lemmas.
227	<code>0 < signing_entropy_token_cardinality.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
228	<code>proof. by rewrite /signing_entropy_token_cardinality. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
229	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
230	<code>lemma signing_entropy_token_distribution_lossless :</code>	States checked lemma <code>signing_entropy_token_distribution_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
231	<code>is_lossless dsigning_entropy_token.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
232	<code>proof.</code>	Begins the proof script for the preceding proposition.
233	<code>rewrite /dsigning_entropy_token.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
234	<code>apply dinter_ll.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
235	<code>by rewrite /signing_entropy_token_cardinality.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
236	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
237	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
238	<code>lemma signing_randomness_source_distribution_lossless :</code>	States checked lemma <code>signing_randomness_source_distribution_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
239	<code>is_lossless dsigning_randomness_source.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
240	<code>proof.</code>	Begins the proof script for the preceding proposition.
241	<code>by rewrite /dsigning_randomness_source;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
242	<code>apply signing_entropy_token_distribution_lossless.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
243	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
244	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
245	<code>lemma signing_coin_distribution_lossless : is_lossless drandom_coins.</code>	States checked lemma <code>signing_coin_distribution_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
246	<code>proof.</code>	Begins the proof script for the preceding proposition.
247	<code>rewrite /drandom_coins /dsigning_randomness.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
248	by apply dmap_ll; apply signing_randomness_source_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
249	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
250	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
251	lemma signing_entropy_token_to_coins_tokenK x :	States checked lemma <code>signing_entropy_token_to_coins_tokenK</code> . The following proof script must derive this proposition from prior definitions and lemmas.
252	signing_entropy_token_of_coins (signing_entropy_token_to_coins x) = x.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
253	proof.	Begins the proof script for the preceding proposition.
254	rewrite /signing_entropy_token_of_coins	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
255	/signing_entropy_token_to_coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
256	rewrite /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
257	smt(divz_eq divz_mul).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
258	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
259	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
260	lemma signing_randomness_from_source_tokenK src :	States checked lemma <code>signing_randomness_from_source_tokenK</code> . The following proof script must derive this proposition from prior definitions and lemmas.
261	signing_entropy_token_of_coins (signing_randomness_from_source src) = src.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
262	proof.	Begins the proof script for the preceding proposition.
263	by rewrite /signing_randomness_from_source	Discharges the current goal in one step using the cited simplification, lemma, or automation.
264	signing_entropy_token_to_coins_tokenK.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
265	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
266	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
267	lemma signing_entropy_token_to_coins_size x :	States checked lemma <code>signing_entropy_token_to_coins_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
268	size (signing_entropy_token_to_coins x) = seedbytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
269	proof.	Begins the proof script for the preceding proposition.
270	rewrite /signing_entropy_token_to_coins /seedbytes.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
271	have hsize : size (nseq (32 - 8) 0) = max (32 - 8) 0 by	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
272	exact size_nseq.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
273	change (8 + size (nseq (32 - 8) 0) = 32).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
274	rewrite hsize.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
275	rewrite /max.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
276	trivial.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
277	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
278	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
279	lemma signing_entropy_token_to_coins_byte_domain x :	States checked lemma <code>signing_entropy_token_to_coins_byte_domain</code> . The following proof script must derive this proposition from prior definitions and lemmas.
280	signing_entropy_token_in_range x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
281	all signing_randomness_byte_ok (signing_entropy_token_to_coins x).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
282	proof.	Begins the proof script for the preceding proposition.
283	rewrite /signing_entropy_token_in_range	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
284	/signing_entropy_token_cardinality	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
285	/signing_entropy_token_to_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
286	/signing_randomness_byte_ok /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
287	move=> xrange.	Introduces universally quantified variables or hypotheses into the proof context.
288	rewrite /seedbytes /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
289	smt(modz_ge0 ltz_pmod divz_ge0 ltz_divLR all_nseq).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
290	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
291	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
292	lemma signing_entropy_token_to_coins_domain x :	States checked lemma <code>signing_entropy_token_to_coins_domain</code> . The following proof script must derive this proposition from prior definitions and lemmas.
293	signing_entropy_token_in_range x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
294	signing_randomness_domain (signing_entropy_token_to_coins x).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
295	proof.	Begins the proof script for the preceding proposition.
296	move=> xrange.	Introduces universally quantified variables or hypotheses into the proof context.
297	rewrite /signing_randomness_domain signing_entropy_token_to_coins_size	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
298	(signing_entropy_token_to_coins_tokenK x).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
299	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
300	split.	Splits a conjunction or structured goal into independent proof obligations.
301	+ by apply signing_entropy_token_to_coins_byte_domain.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
302	by apply xrange.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
303	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
304	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
305	lemma signing_randomness_from_source_domain src :	States checked lemma <code>signing_randomness_from_source_domain</code> . The following proof script must derive this proposition from prior definitions and lemmas.
306	signing_randomness_source_valid src =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
307	signing_randomness_domain (signing_randomness_from_source src).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
308	proof.	Begins the proof script for the preceding proposition.
309	by rewrite /signing_randomness_source_valid	Discharges the current goal in one step using the cited simplification, lemma, or automation.
310	/signing_randomness_from_source;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
311	apply signing_entropy_token_to_coins_domain.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
312	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
313	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
314	lemma signing_entropy_token_distribution_point_bound x :	States checked lemma <code>signing_entropy_token_distribution_point_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
315	mu dsigning_entropy_token (pred1 x) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
316	signing_randomness_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
317	proof.	Begins the proof script for the preceding proposition.
318	rewrite /signing_randomness_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
319	rewrite /dsigning_entropy_token dinter1E /pred1	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
320	/signing_entropy_token_cardinality.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
321	case: (0 <= x <= 288230376151711744 - 1).	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
322	+ by move=> _; rewrite RField.div1r.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
323	by move=> _; apply divr_ge0; [trivial trivial].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
324	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
325	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
326	lemma signing_randomness_source_distribution_point_bound src :	States checked lemma signing_randomness_source_distribution_point_bound. The following proof script must derive this proposition from prior definitions and lemmas.
327	mu dsigning_randomness_source (pred1 src) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
328	signing_randomness_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
329	proof.	Begins the proof script for the preceding proposition.
330	by rewrite /dsigning_randomness_source;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
331	apply signing_entropy_token_distribution_point_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
332	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
333	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
334	lemma signing_coin_distribution_token_point_bound token :	States checked lemma signing_coin_distribution_token_point_bound. The following proof script must derive this proposition from prior definitions and lemmas.
335	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
336	(fun coins => signing_entropy_token_of_coins coins = token) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
337	signing_randomness_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
338	proof.	Begins the proof script for the preceding proposition.
339	rewrite /drandom_coins /dsigning_randomness dmapE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
340	apply (1er_trans (mu dsigning_entropy_token (pred1 token))).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
341	+ apply mu_le => x _ token_eq.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
342	rewrite /pred1 /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
343	rewrite -token_eq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
344	by rewrite (signing_entropy_token_to_coins_tokenK x).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
345	by apply signing_entropy_token_distribution_point_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
346	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
347	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
348	lemma signing_coin_distribution_token_spread :	States checked lemma signing_coin_distribution_token_spread. The following proof script must derive this proposition from prior definitions and lemmas.
349	signing_randomness_token_spread drandom_coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
350	proof.	Begins the proof script for the preceding proposition.
351	rewrite /signing_randomness_token_spread.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
352	by apply signing_coin_distribution_token_point_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
353	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
354	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
355	lemma signing_coin_distribution_domain coins :	States checked lemma signing_coin_distribution_domain. The following proof script must derive this proposition from prior definitions and lemmas.
356	coins \in drandom_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
357	signing_randomness_domain coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
358	proof.	Begins the proof script for the preceding proposition.
359	rewrite /drandom_coins /dsigning_randomness	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
360	/dsigning_randomness_source /signing_randomness_from_source.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
361	rewrite supp_dmap.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
362	move=> [src [src_supp ->]].	Introduces universally quantified variables or hypotheses into the proof context.
363	apply signing_entropy_token_to_coins_domain.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
364	move: src_supp.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
365	rewrite /dsigning_entropy_token supp_dinter	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
366	/signing_entropy_token_in_range	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
367	/signing_entropy_token_cardinality.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
368	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
369	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
370	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
371	lemma signing_coin_distribution_domain_full :	States checked lemma signing_coin_distribution_domain_full. The following proof script must derive this proposition from prior definitions and lemmas.
372	mu drandom_coins signing_randomness_domain = 1%r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
373	proof.	Begins the proof script for the preceding proposition.
374	apply eq1_mu.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
375	+ by apply signing_coin_distribution_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
376	by apply signing_coin_distribution_domain.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
377	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
378	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
379	lemma signing_coin_distribution_ok :	States checked lemma signing_coin_distribution_ok. The following proof script must derive this proposition from prior definitions and lemmas.
380	signing_randomness_distribution_ok drandom_coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
381	proof.	Begins the proof script for the preceding proposition.
382	rewrite /signing_randomness_distribution_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
383	split.	Splits a conjunction or structured goal into independent proof obligations.
384	+ by apply signing_coin_distribution_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
385	split.	Splits a conjunction or structured goal into independent proof obligations.
386	+ by apply signing_coin_distribution_domain.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
387	by apply signing_coin_distribution_token_spread.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
388	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
389	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
390	lemma coeff_distribution_lossless : is_lossless dcoeff.	States checked lemma coeff_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
391	proof. by rewrite /dcoeff; apply dinter_ll. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
392	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
393	lemma poly_distribution_lossless : is_lossless dpoly.	States checked lemma poly_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
394	proof.	Begins the proof script for the preceding proposition.
395	rewrite /dpoly.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
396	by apply dlist_ll; apply coeff_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
397	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
398	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
399	lemma polyveck_distribution_lossless md : is_lossless (dpolyveck md).	States checked lemma polyveck_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
400	proof.	Begins the proof script for the preceding proposition.
401	rewrite /dpolyveck.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
402	by apply dlist_ll; apply poly_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
403	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
404	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
405	lemma polyvecl_distribution_lossless md : is_lossless (dpolyvecl md).	States checked lemma polyvecl_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
406	proof.	Begins the proof script for the preceding proposition.
407	rewrite /dpolyvecl.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
408	by apply dlist_ll; apply poly_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
409	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
410	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
411	lemma matrix_distribution_lossless md : is_lossless (dmatrix md).	States checked lemma matrix_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
412	proof.	Begins the proof script for the preceding proposition.
413	rewrite /dmatrix.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
414	by apply dlist_ll; apply polyvecl_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
415	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
416	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
417	lemma challenge_distribution_lossless md : is_lossless (dchallenge md).	States checked lemma challenge_distribution_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
418	proof.	Begins the proof script for the preceding proposition.
419	rewrite /dchallenge.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
420	by apply dmap_ll; apply crh_distribution_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
421	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
422	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
423	lemma signing_sample_pair_of_coins_wf md sk m ctx coins :	States checked lemma signing_sample_pair_of_coins_wf. The following proof script must derive this proposition from prior definitions and lemmas.
424	signing_sample_pair_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
425	(signing_sample_pair_of_coins md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
426	proof.	Begins the proof script for the preceding proposition.
427	rewrite /signing_sample_pair_wf /signing_sample_pair_of_coins	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
428	/signing_sample_pair_y1 /signing_sample_pair_y2 /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
429	split.	Splits a conjunction or structured goal into independent proof obligations.
430	+ by apply signing_sample_y1_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
431	by apply signing_sample_y2_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
432	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
433	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
434	lemma signing_sample_pair_of_coins_unit md sk m ctx coins :	States checked lemma signing_sample_pair_of_coins_unit. The following proof script must derive this proposition from prior definitions and lemmas.
435	signing_sample_pair_unit md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
436	(signing_sample_pair_of_coins md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
437	proof.	Begins the proof script for the preceding proposition.
438	rewrite /signing_sample_pair_unit /signing_sample_pair_of_coins	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
439	/signing_sample_pair_y1 /signing_sample_pair_y2 /=..	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
440	split.	Splits a conjunction or structured goal into independent proof obligations.
441	+ by apply signing_sample_y1_unit.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
442	by apply signing_sample_y2_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
443	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
444	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
445	lemma signing_sample_bound_gt0 md :	States checked lemma signing_sample_bound_gt0. The following proof script must derive this proposition from prior definitions and lemmas.
446	0 < signing_sample_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
447	proof. by rewrite /signing_sample_bound; apply mode_ln_gt0. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
448	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
449	lemma unit_coeff_bounded_by_signing_sample_bound md c :	States checked lemma unit_coeff_bounded_by_signing_sample_bound. The following proof script must derive this proposition from prior definitions and lemmas.
450	unit_coeff_ok c =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
451	coeff_bounded_by (signing_sample_bound md) c.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
452	proof.	Begins the proof script for the preceding proposition.
453	rewrite /unit_coeff_ok /coeff_bounded_by /signing_sample_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
454	by case md; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
455	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
456	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
457	lemma poly_unit_bounded_by_signing_sample_bound md p :	States checked lemma poly_unit_bounded_by_signing_sample_bound. The following proof script must derive this proposition from prior definitions and lemmas.
458	poly_unit p =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
459	poly_bounded_by (signing_sample_bound md) p.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
460	proof.	Begins the proof script for the preceding proposition.
461	rewrite /poly_unit /poly_bounded_by => -[p_wf p_all].	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
462	split=> // i i_rng.	Splits a conjunction or structured goal into independent proof obligations.
463	move: p_all; rewrite allP => p_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
464	apply unit_coeff_bounded_by_signing_sample_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
465	apply p_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
466	rewrite /poly_coeff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
467	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
468	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
469	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
470	lemma polyvecl_unit_bounded_by_signing_sample_bound md y1 :	States checked lemma <code>polyvecl_unit_bounded_by_signing_sample_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
471	polyvecl_unit md y1 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
472	polyvecl_bounded_by md (signing_sample_bound md) y1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
473	proof.	Begins the proof script for the preceding proposition.
474	rewrite /polyvecl_unit /polyvecl_bounded_by => -[y1_wf y1_all].	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
475	split=> // row row_rng.	Splits a conjunction or structured goal into independent proof obligations.
476	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
477	apply poly_unit_bounded_by_signing_sample_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
478	apply y1_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
479	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
480	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
481	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
482	lemma polyveck_unit_bounded_by_signing_sample_bound md y2 :	States checked lemma <code>polyveck_unit_bounded_by_signing_sample_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
483	polyveck_unit md y2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
484	polyveck_bounded_by md (signing_sample_bound md) y2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
485	proof.	Begins the proof script for the preceding proposition.
486	rewrite /polyveck_unit /polyveck_bounded_by => -[y2_wf y2_all].	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
487	split=> // row row_rng.	Splits a conjunction or structured goal into independent proof obligations.
488	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
489	apply poly_unit_bounded_by_signing_sample_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
490	apply y2_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
491	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
492	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
493	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
494	lemma signing_sample_pair_unit_bounded md s :	States checked lemma <code>signing_sample_pair_unit_bounded</code> . The following proof script must derive this proposition from prior definitions and lemmas.
495	signing_sample_pair_unit md s =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
496	signing_sample_pair_bounded md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
497	proof.	Begins the proof script for the preceding proposition.
498	rewrite /signing_sample_pair_unit /signing_sample_pair_bounded.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
499	move=> [y1_unit y2_unit].	Introduces universally quantified variables or hypotheses into the proof context.
500	split.	Splits a conjunction or structured goal into independent proof obligations.
501	+ by apply polyvecl_unit_bounded_by_signing_sample_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
502	by apply polyveck_unit_bounded_by_signing_sample_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
503	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
504	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
505	lemma signing_sample_pair_of_coins_sample_ok md sk m ctx coins :	States checked lemma <code>signing_sample_pair_of_coins_sample_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
506	signing_sample_pair_sample_ok md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
507	(signing_sample_pair_of_coins md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
508	proof.	Begins the proof script for the preceding proposition.
509	rewrite /signing_sample_pair_sample_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
510	split.	Splits a conjunction or structured goal into independent proof obligations.
511	+ by apply signing_sample_pair_of_coins_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
512	apply signing_sample_pair_unit_bounded.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
513	by apply signing_sample_pair_of_coins_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
514	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
515	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
516	lemma structural_signing_sample_pair_lossless md sk m ctx coins :	States checked lemma <code>structural_signing_sample_pair_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
517	is_lossless	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
518	(dstructural_signing_sample_pair md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
519	proof.	Begins the proof script for the preceding proposition.
520	by rewrite /dstructural_signing_sample_pair; apply dunit_ll.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
521	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
522	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
523	lemma structural_signing_sample_pair_support md sk m ctx coins s :	States checked lemma <code>structural_signing_sample_pair_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
524	s \in dstructural_signing_sample_pair md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
525	signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
526	proof.	Begins the proof script for the preceding proposition.
527	rewrite /dstructural_signing_sample_pair supp_dunit.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
528	move=> ->.	Introduces universally quantified variables or hypotheses into the proof context.
529	split.	Splits a conjunction or structured goal into independent proof obligations.
530	+ by apply signing_sample_pair_of_coins_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
531	by apply signing_sample_pair_of_coins_unit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
532	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
533	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
534	lemma structural_signing_sample_pair_sample_ok md sk m ctx coins s :	States checked lemma <code>structural_signing_sample_pair_sample_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
535	s \in dstructural_signing_sample_pair md sk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
536	signing_sample_pair_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
537	proof.	Begins the proof script for the preceding proposition.
538	rewrite /dstructural_signing_sample_pair supp_dunit.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
539	move=> ->.	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
540	by apply signing_sample_pair_of_coins_sample_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
541	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
542	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
543	lemma structural_signing_sample_pair_distribution_ok md sk m ctx coins :	States checked lemma structural_signing_sample_pair_distribution_ok. The following proof script must derive this proposition from prior definitions and lemmas.
544	signing_sample_pair_distribution_ok md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
545	(dstructural_signing_sample_pair md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
546	proof.	Begins the proof script for the preceding proposition.
547	rewrite /signing_sample_pair_distribution_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
548	split.	Splits a conjunction or structured goal into independent proof obligations.
549	+ by apply structural_signing_sample_pair_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
550	by move=> s; apply structural_signing_sample_pair_sample_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
551	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
552	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
553	lemma signing_unit_coeff_lossless :	States checked lemma signing_unit_coeff_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
554	is_lossless dsigning_unit_coeff.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
555	proof.	Begins the proof script for the preceding proposition.
556	rewrite /dsigning_unit_coeff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
557	by apply duniform_ll.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
558	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
559	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
560	lemma signing_unit_coeff_support c :	States checked lemma signing_unit_coeff_support. The following proof script must derive this proposition from prior definitions and lemmas.
561	c \in dsigning_unit_coeff =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
562	unit_coeff_ok c.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
563	proof.	Begins the proof script for the preceding proposition.
564	rewrite /dsigning_unit_coeff supp_duniform /= /unit_coeff_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
565	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
566	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
567	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
568	lemma signing_bounded_coeff_cardinality_positive md :	States checked lemma signing_bounded_coeff_cardinality_positive. The following proof script must derive this proposition from prior definitions and lemmas.
569	0 < signing_bounded_coeff_cardinality md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
570	proof.	Begins the proof script for the preceding proposition.
571	rewrite /signing_bounded_coeff_cardinality.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
572	by smt(signing_sample_bound_gt0).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
573	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
574	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
575	lemma signing_bounded_coeff_lossless md :	States checked lemma signing_bounded_coeff_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
576	is_lossless (dsigning_bounded_coeff md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
577	<code>proof.</code>	Begins the proof script for the preceding proposition.
578	<code>rewrite /dsigning_bounded_coeff.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
579	<code>apply dinter_ll.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
580	<code>by smt(signing_sample_bound_gt0).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
581	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
582	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
583	<code>lemma signing_bounded_coeff_support md c :</code>	States checked lemma <code>signing_bounded_coeff_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
584	<code>c \in dsigning_bounded_coeff md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
585	<code>coeff_bounded_by (signing_sample_bound md) c.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
586	<code>proof.</code>	Begins the proof script for the preceding proposition.
587	<code>by rewrite /dsigning_bounded_coeff supp_dinter /coeff_bounded_by.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
588	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
589	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
590	<code>lemma signing_bounded_coeff_supportP md c :</code>	States checked lemma <code>signing_bounded_coeff_supportP</code> . The following proof script must derive this proposition from prior definitions and lemmas.
591	<code>c \in dsigning_bounded_coeff md <=></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
592	<code>coeff_bounded_by (signing_sample_bound md) c.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
593	<code>proof.</code>	Begins the proof script for the preceding proposition.
594	<code>by rewrite /dsigning_bounded_coeff supp_dinter /coeff_bounded_by.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
595	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
596	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
597	<code>lemma signing_bounded_coeff_point_bound md c :</code>	States checked lemma <code>signing_bounded_coeff_point_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
598	<code>mu (dsigning_bounded_coeff md) (pred1 c) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
599	<code>signing_bounded_coeff_point_bound md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
600	<code>proof.</code>	Begins the proof script for the preceding proposition.
601	<code>rewrite /dsigning_bounded_coeff /signing_bounded_coeff_point_bound</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
602	<code>/signing_bounded_coeff_cardinality /signing_sample_bound</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
603	<code>dinter1E /pred1.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
604	<code>case md; rewrite /mode_ln /=.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
605	<code>+ case: (-8192 <= c <= 8192) => _.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
606	<code>+ by trivial.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
607	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
608	<code>+ case: (-8192 <= c <= 8192) => _.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
609	<code>+ by trivial.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
610	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
611	<code>case: (-8192 <= c <= 8192) => _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
612	<code>+ by trivial.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
613	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
614	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
615	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
616	lemma signing_bounded_coeff_point_bound_nonnegative md :	States checked lemma signing_bounded_coeff_point_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
617	0%r <= signing_bounded_coeff_point_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
618	proof.	Begins the proof script for the preceding proposition.
619	rewrite /signing_bounded_coeff_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
620	apply divr_ge0.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
621	+ by trivial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
622	by smt(signing_bounded_coeff_cardinality_positive).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
623	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
624	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
625	lemma dlist_size_point_bound ['a] (d : 'a distr) bd xs :	States checked lemma dlist_size_point_bound. The following proof script must derive this proposition from prior definitions and lemmas.
626	0%r <= bd =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
627	(forall x, mu d (pred1 x) <= bd) =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
628	mu (dlist d (size xs)) (pred1 xs) <= bd ^ (size xs).	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
629	proof.	Begins the proof script for the preceding proposition.
630	move=> bd_ge0 point_bound.	Introduces universally quantified variables or hypotheses into the proof context.
631	rewrite dlist1E 1:size_ge0 /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
632	elim xs=> [x xs ih].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
633	+ by rewrite StdBigop.Bigreal.BRM.big_nil /= RField.expr0.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
634	rewrite StdBigop.Bigreal.BRM.big_cons /predT /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
635	rewrite (_ : 1 + size xs = size xs + 1) 1:/#.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
636	rewrite RField.exprS 1:size_ge0.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
637	apply ler_pmul.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
638	+ by apply ge0_mu.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
639	+ apply (StdBigop.Bigreal.BRM.big_ind (fun z => 0%r <= z)).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
640	+ by move=> a b; apply mulr_ge0.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
641	+ by trivial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
642	by move=> z _; apply ge0_mu.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
643	+ by apply point_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
644	by apply ih.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
645	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
646	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
647	lemma dlist_point_bound ['a] (d : 'a distr) len bd xs :	States checked lemma dlist_point_bound. The following proof script must derive this proposition from prior definitions and lemmas.
648	0 <= len =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
649	<code>0%r <= bd =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
650	<code>(forall x, mu d (pred1 x) <= bd) =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
651	<code>mu (dlist d len) (pred1 xs) <= bd ^ len.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
652	<code>proof.</code>	Begins the proof script for the preceding proposition.
653	<code>move=> len_ge0 bd_ge0 point_bound.</code>	Introduces universally quantified variables or hypotheses into the proof context.
654	<code>case: (len = size xs) => lenE.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
655	<code>+ rewrite lenE.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
656	<code>by apply dlist_size_point_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
657	<code>rewrite dlist1E // lenE.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
658	<code>by apply expr_ge0.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
659	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
660	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
661	<code>lemma hyperball_coeff_point_bound_nonnegative md :</code>	States checked lemma <code>hyperball_coeff_point_bound_nonnegative</code> . The following proof script must derive this proposition from prior definitions and lemmas.
662	<code>0%r <= hyperball_coeff_point_bound md.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
663	<code>proof.</code>	Begins the proof script for the preceding proposition.
664	<code>by rewrite /hyperball_coeff_point_bound;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
665	<code>apply signing_bounded_coeff_point_bound_nonnegative.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
666	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
667	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
668	<code>lemma hyperball_coeff_point_boundE md c :</code>	States checked lemma <code>hyperball_coeff_point_boundE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
669	<code>mu (dhyperball_coeff md) (pred1 c) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
670	<code>hyperball_coeff_point_bound md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
671	<code>proof.</code>	Begins the proof script for the preceding proposition.
672	<code>by rewrite /dhyperball_coeff /hyperball_coeff_point_bound;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
673	<code>apply signing_bounded_coeff_point_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
674	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
675	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
676	<code>lemma signing_unit_poly_lossless :</code>	States checked lemma <code>signing_unit_poly_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
677	<code>is_lossless dsigning_unit_poly.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
678	<code>proof.</code>	Begins the proof script for the preceding proposition.
679	<code>rewrite /dsigning_unit_poly.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
680	<code>by apply dlist_ll; apply signing_unit_coeff_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
681	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
682	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
683	<code>lemma signing_unit_poly_support p :</code>	States checked lemma <code>signing_unit_poly_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
684	<code>p \in dsigning_unit_poly =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
685	<code>poly_unit p.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
686	proof.	Begins the proof script for the preceding proposition.
687	rewrite /dsigning_unit_poly supp_dlist; first by smt(n_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
688	move=> [p_sz p_all].	Introduces universally quantified variables or hypotheses into the proof context.
689	rewrite /poly_unit /poly_wf p_sz /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
690	apply/allP => c c_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
691	move: p_all; rewrite allP => p_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
692	by apply signing_unit_coeff_support; apply p_all.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
693	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
694	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
695	lemma signing_bounded_poly_lossless md :	States checked lemma signing_bounded_poly_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
696	is_lossless (dsigning_bounded_poly md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
697	proof.	Begins the proof script for the preceding proposition.
698	rewrite /dsigning_bounded_poly.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
699	by apply dlist_ll; apply signing_bounded_coeff_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
700	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
701	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
702	lemma signing_bounded_poly_support md p :	States checked lemma signing_bounded_poly_support. The following proof script must derive this proposition from prior definitions and lemmas.
703	p \in dsigning_bounded_poly md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
704	poly_bounded_by (signing_sample_bound md) p.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
705	proof.	Begins the proof script for the preceding proposition.
706	rewrite /dsigning_bounded_poly supp_dlist; first by smt(n_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
707	move=> [p_sz p_all].	Introduces universally quantified variables or hypotheses into the proof context.
708	rewrite /poly_bounded_by /poly_wf p_sz /.=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
709	move: p_all; rewrite allP => p_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
710	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
711	apply signing_bounded_coeff_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
712	apply p_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
713	rewrite /poly_coeff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
714	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
715	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
716	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
717	lemma signing_bounded_poly_supportP md p :	States checked lemma signing_bounded_poly_supportP. The following proof script must derive this proposition from prior definitions and lemmas.
718	p \in dsigning_bounded_poly md <=>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
719	poly_bounded_by (signing_sample_bound md) p.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
720	proof.	Begins the proof script for the preceding proposition.
721	rewrite /dsigning_bounded_poly supp_dlist; first by smt(n_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
722	rewrite /poly_bounded_by /poly_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
723	split.	Splits a conjunction or structured goal into independent proof obligations.
724	+ move=> [p_sz p_all]; split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
725	move: p_all; rewrite allP => p_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
726	move=> i i_rng.	Introduces universally quantified variables or hypotheses into the proof context.
727	apply signing_bounded_coeff_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
728	apply p_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
729	rewrite /poly_coeff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
730	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
731	move=> [p_sz p_bound].	Introduces universally quantified variables or hypotheses into the proof context.
732	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
733	rewrite allP => c c_mem.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
734	rewrite signing_bounded_coeff_supportP.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
735	by smt(nthP).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
736	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
737	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
738	lemma hyperball_poly_point_bound_nonnegative md :	States checked lemma hyperball_poly_point_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
739	0%r <= hyperball_poly_point_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
740	proof.	Begins the proof script for the preceding proposition.
741	rewrite /hyperball_poly_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
742	by apply expr_ge0; apply hyperball_coeff_point_bound_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
743	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
744	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
745	lemma hyperball_poly_lossless md :	States checked lemma hyperball_poly_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
746	is_lossless (dhyperball_poly md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
747	proof.	Begins the proof script for the preceding proposition.
748	by rewrite /dhyperball_poly; apply signing_bounded_poly_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
749	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
750	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
751	lemma hyperball_poly_support md p :	States checked lemma hyperball_poly_support. The following proof script must derive this proposition from prior definitions and lemmas.
752	p \in dhyperball_poly md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
753	hyperball_poly_ok md p.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
754	proof.	Begins the proof script for the preceding proposition.
755	by rewrite /dhyperball_poly /hyperball_poly_ok;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
756	apply signing_bounded_poly_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
757	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
758	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
759	lemma hyperball_poly_supportP md p :	States checked lemma hyperball_poly_supportP. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
760	<code>p \in dhyperball_poly md <=> hyperball_poly_ok md p.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
761	<code>proof.</code>	Begins the proof script for the preceding proposition.
762	<code>by rewrite /dhyperball_poly /hyperball_poly_ok</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
763	<code>signing_bounded_poly_supportP.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
764	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
765	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
766	<code>lemma hyperball_poly_point_boundE md p :</code>	States checked lemma <code>hyperball_poly_point_boundE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
767	<code>mu (dhyperball_poly md) (pred1 p) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
768	<code>hyperball_poly_point_bound md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
769	<code>proof.</code>	Begins the proof script for the preceding proposition.
770	<code>rewrite /dhyperball_poly /dsigning_bounded_poly</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
771	<code>/hyperball_poly_point_bound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
772	<code>apply dlist_point_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
773	<code>+ by smt(n_gt0).</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
774	<code>+ by apply hyperball_coeff_point_bound_nonnegative.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
775	<code>by move=> c; apply hyperball_coeff_point_boundE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
776	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
777	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
778	<code>lemma signing_unit_polyvecl_lossless md :</code>	States checked lemma <code>signing_unit_polyvecl_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
779	<code>is_lossless (dsigning_unit_polyvecl md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
780	<code>proof.</code>	Begins the proof script for the preceding proposition.
781	<code>rewrite /dsigning_unit_polyvecl.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
782	<code>by apply dlist_ll; apply signing_unit_poly_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
783	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
784	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
785	<code>lemma signing_unit_polyveck_lossless md :</code>	States checked lemma <code>signing_unit_polyveck_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
786	<code>is_lossless (dsigning_unit_polyveck md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
787	<code>proof.</code>	Begins the proof script for the preceding proposition.
788	<code>rewrite /dsigning_unit_polyveck.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
789	<code>by apply dlist_ll; apply signing_unit_poly_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
790	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
791	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
792	<code>lemma signing_bounded_polyvecl_lossless md :</code>	States checked lemma <code>signing_bounded_polyvecl_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
793	<code>is_lossless (dsigning_bounded_polyvecl md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
794	<code>proof.</code>	Begins the proof script for the preceding proposition.
795	<code>rewrite /dsigning_bounded_polyvecl.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
796	<code>by apply dlist_ll; apply signing_bounded_poly_lossless.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
797	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
798	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
799	lemma signing_bounded_polyveck_lossless md :	States checked lemma <code>signing_bounded_polyveck_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
800	is_lossless (dsigning_bounded_polyveck md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
801	proof.	Begins the proof script for the preceding proposition.
802	rewrite /dsigning_bounded_polyveck.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
803	by apply dlist_ll; apply signing_bounded_poly_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
804	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
805	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
806	lemma signing_unit_polyveck1_support md y1 :	States checked lemma <code>signing_unit_polyveck1_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
807	y1 \in dsigning_unit_polyveck1 md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
808	polyveck1_unit md y1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
809	proof.	Begins the proof script for the preceding proposition.
810	rewrite /dsigning_unit_polyveck1 supp_dlist; first by smt(mode_l_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
811	move=> [y1_sz y1_all].	Introduces universally quantified variables or hypotheses into the proof context.
812	rewrite /polyveck1_unit /polyveck1_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
813	split.	Splits a conjunction or structured goal into independent proof obligations.
814	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
815	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
816	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
817	have p_supp : p \in dsigning_unit_poly by apply y1_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
818	move: (signing_unit_poly_support p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
819	by rewrite /poly_unit => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
820	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
821	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
822	by apply signing_unit_poly_support; apply y1_all.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
823	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
824	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
825	lemma signing_unit_polyveck_support md y2 :	States checked lemma <code>signing_unit_polyveck_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
826	y2 \in dsigning_unit_polyveck md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
827	polyveck_unit md y2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
828	proof.	Begins the proof script for the preceding proposition.
829	rewrite /dsigning_unit_polyveck supp_dlist; first by smt(mode_k_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
830	move=> [y2_sz y2_all].	Introduces universally quantified variables or hypotheses into the proof context.
831	rewrite /polyveck_unit /polyveck_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
832	split.	Splits a conjunction or structured goal into independent proof obligations.
833	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
834	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
835	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
836	have p_supp : p \in dsigning_unit_poly by apply y2_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
837	move: (signing_unit_poly_support p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
838	by rewrite /poly_unit => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
839	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
840	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
841	by apply signing_unit_poly_support; apply y2_all.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
842	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
843	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
844	lemma signing_bounded_polyvec1_support md y1 :	States checked lemma signing_bounded_polyvec1_support. The following proof script must derive this proposition from prior definitions and lemmas.
845	y1 \in dsigning_bounded_polyvec1 md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
846	polyvec1_bounded_by md (signing_sample_bound md) y1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
847	proof.	Begins the proof script for the preceding proposition.
848	rewrite /dsigning_bounded_polyvec1 supp_dlist; first by smt(mode_l_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
849	move=> [y1_sz y1_all].	Introduces universally quantified variables or hypotheses into the proof context.
850	rewrite /polyvec1_bounded_by /polyvec1_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
851	split.	Splits a conjunction or structured goal into independent proof obligations.
852	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
853	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
854	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
855	have p_supp : p \in dsigning_bounded_poly md by apply y1_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
856	move: (signing_bounded_poly_support md p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
857	by rewrite /poly_bounded_by => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
858	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
859	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
860	apply signing_bounded_poly_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
861	apply y1_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
862	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
863	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
864	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
865	lemma signing_bounded_polyvec1_supportP md y1 :	States checked lemma signing_bounded_polyvec1_supportP. The following proof script must derive this proposition from prior definitions and lemmas.
866	y1 \in dsigning_bounded_polyvec1 md <=>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
867	polyvec1_bounded_by md (signing_sample_bound md) y1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
868	proof.	Begins the proof script for the preceding proposition.
869	rewrite /dsigning_bounded_polyvec1 supp_dlist; first by smt(mode_l_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
870	rewrite /polyvec1_bounded_by /polyvec1_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
871	split.	Splits a conjunction or structured goal into independent proof obligations.
872	+ move=> [y1_sz y1_all].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
873	split.	Splits a conjunction or structured goal into independent proof obligations.
874	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
875	rewrite allP => p p_mem.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
876	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
877	have p_supp : p \in dsigning_bounded_poly md by apply y1_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
878	move: (signing_bounded_poly_support md p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
879	by rewrite /poly_bounded_by => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
880	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
881	move: y1_all; rewrite allP => y1_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
882	apply signing_bounded_poly_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
883	apply y1_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
884	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
885	move=> [[y1_sz y1_allwf] y1_bound].	Introduces universally quantified variables or hypotheses into the proof context.
886	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
887	rewrite allP => p p_mem.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
888	rewrite signing_bounded_poly_supportP.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
889	by smt(nthP).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
890	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
891	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
892	lemma signing_bounded_polyveck_support md y2 :	States checked lemma signing_bounded_polyveck_support. The following proof script must derive this proposition from prior definitions and lemmas.
893	y2 \in dsigning_bounded_polyveck md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
894	polyveck_bounded_by md (signing_sample_bound md) y2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
895	proof.	Begins the proof script for the preceding proposition.
896	rewrite /dsigning_bounded_polyveck supp_dlist; first by smt(mode_k_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
897	move=> [y2_sz y2_all].	Introduces universally quantified variables or hypotheses into the proof context.
898	rewrite /polyveck_bounded_by /polyveck_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
899	split.	Splits a conjunction or structured goal into independent proof obligations.
900	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
901	apply/allP => p p_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
902	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
903	have p_supp : p \in dsigning_bounded_poly md by apply y2_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
904	move: (signing_bounded_poly_support md p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
905	by rewrite /poly_bounded_by => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
906	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
907	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
908	apply signing_bounded_poly_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
909	apply y2_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
910	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
911	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
912	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
913	lemma signing_bounded_polyveck_supportP md y2 :	States checked lemma signing_bounded_polyveck_supportP. The following proof script must derive this proposition from prior definitions and lemmas.
914	y2 \in dsigning_bounded_polyveck md <=>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
915	polyveck_bounded_by md (signing_sample_bound md) y2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
916	proof.	Begins the proof script for the preceding proposition.
917	rewrite /dsigning_bounded_polyveck supp_dlist; first by smt(mode_k_gt0).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
918	rewrite /polyveck_bounded_by /polyveck_wf.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
919	split.	Splits a conjunction or structured goal into independent proof obligations.
920	+ move=> [y2_sz y2_all].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
921	split.	Splits a conjunction or structured goal into independent proof obligations.
922	+ split=> //.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
923	rewrite allP => p p_mem.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
924	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
925	have p_supp : p \in dsigning_bounded_poly md by apply y2_all.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
926	move: (signing_bounded_poly_support md p p_supp).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
927	by rewrite /poly_bounded_by => -[p_wf _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
928	move=> row row_rng.	Introduces universally quantified variables or hypotheses into the proof context.
929	move: y2_all; rewrite allP => y2_all.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
930	apply signing_bounded_poly_support.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
931	apply y2_all.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
932	by apply mem_nth; smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
933	move=> [[y2_sz y2_allwf] y2_bound].	Introduces universally quantified variables or hypotheses into the proof context.
934	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
935	rewrite allP => p p_mem.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
936	rewrite signing_bounded_poly_supportP.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
937	by smt(nthP).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
938	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
939	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
940	lemma hyperball_polyvecl_point_bound_nonnegative md :	States checked lemma hyperball_polyvecl_point_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
941	0%r <= hyperball_polyvecl_point_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
942	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
943	rewrite /hyperball_polyvecl_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
944	by apply expr_ge0; apply hyperball_poly_point_bound_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
945	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
946	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
947	lemma hyperball_polyveck_point_bound_nonnegative md :	States checked lemma hyperball_polyveck_point_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
948	0%r <= hyperball_polyveck_point_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
949	proof.	Begins the proof script for the preceding proposition.
950	rewrite /hyperball_polyveck_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
951	by apply expr_ge0; apply hyperball_poly_point_bound_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
952	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
953	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
954	lemma hyperball_sample_pair_point_bound_nonnegative md :	States checked lemma hyperball_sample_pair_point_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
955	0%r <= hyperball_sample_pair_point_bound md.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
956	proof.	Begins the proof script for the preceding proposition.
957	rewrite /hyperball_sample_pair_point_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
958	by apply mulr_ge0;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
959	[apply hyperball_polyvecl_point_bound_nonnegative	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
960	apply hyperball_polyveck_point_bound_nonnegative].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
961	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
962	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
963	lemma hyperball_polyvecl_lossless md :	States checked lemma hyperball_polyvecl_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
964	is_lossless (dhyperball_polyvecl md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
965	proof.	Begins the proof script for the preceding proposition.
966	by rewrite /dhyperball_polyvecl; apply signing_bounded_polyvecl_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
967	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
968	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
969	lemma hyperball_polyveck_lossless md :	States checked lemma hyperball_polyveck_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
970	is_lossless (dhyperball_polyveck md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
971	proof.	Begins the proof script for the preceding proposition.
972	by rewrite /dhyperball_polyveck; apply signing_bounded_polyveck_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
973	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
974	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
975	lemma hyperball_polyvecl_support md y1 :	States checked lemma hyperball_polyvecl_support. The following proof script must derive this proposition from prior definitions and lemmas.
976	y1 \in dhyperball_polyvecl md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
977	hyperball_polyvecl_ok md y1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
978	proof.	Begins the proof script for the preceding proposition.
979	by rewrite /dhyperball_polyvecl /hyperball_polyvecl_ok;	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
980	<code>apply signing_bounded_polyvecl_support.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
981	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
982	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
983	<code>lemma hyperball_polyvecl_supportP md y1 :</code>	States checked lemma <code>hyperball_polyvecl_supportP</code> . The following proof script must derive this proposition from prior definitions and lemmas.
984	<code>y1 \in dhyperball_polyvecl md <=> hyperball_polyvecl_ok md y1.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
985	<code>proof.</code>	Begins the proof script for the preceding proposition.
986	<code>by rewrite /dhyperball_polyvecl /hyperball_polyvecl_ok</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
987	<code>signing_bounded_polyvecl_supportP.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
988	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
989	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
990	<code>lemma hyperball_polyveck_support md y2 :</code>	States checked lemma <code>hyperball_polyveck_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
991	<code>y2 \in dhyperball_polyveck md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
992	<code>hyperball_polyveck_ok md y2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
993	<code>proof.</code>	Begins the proof script for the preceding proposition.
994	<code>by rewrite /dhyperball_polyveck /hyperball_polyveck_ok;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
995	<code>apply signing_bounded_polyveck_support.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
996	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
997	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
998	<code>lemma hyperball_polyveck_supportP md y2 :</code>	States checked lemma <code>hyperball_polyveck_supportP</code> . The following proof script must derive this proposition from prior definitions and lemmas.
999	<code>y2 \in dhyperball_polyveck md <=> hyperball_polyveck_ok md y2.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1000	<code>proof.</code>	Begins the proof script for the preceding proposition.
1001	<code>by rewrite /dhyperball_polyveck /hyperball_polyveck_ok</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1002	<code>signing_bounded_polyveck_supportP.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1003	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1004	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1005	<code>lemma hyperball_polyvecl_point_boundE md y1 :</code>	States checked lemma <code>hyperball_polyvecl_point_boundE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1006	<code>mu (dhyperball_polyvecl md) (pred1 y1) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1007	<code>hyperball_polyvecl_point_bound md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1008	<code>proof.</code>	Begins the proof script for the preceding proposition.
1009	<code>rewrite /dhyperball_polyvecl /dsigning_bounded_polyvecl</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1010	<code>/hyperball_polyvecl_point_bound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1011	<code>apply dlist_point_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1012	<code>+ by smt(mode_l_gt0).</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1013	<code>+ by apply hyperball_poly_point_bound_nonnegative.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1014	<code>by move=> p; apply hyperball_poly_point_boundE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1015	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1016	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1017	<code>lemma hyperball_polyveck_point_boundE md y2 :</code>	States checked lemma <code>hyperball_polyveck_point_boundE</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1018	mu (dhyperball_polyveck md) (pred1 y2) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1019	hyperball_polyveck_point_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1020	proof.	Begins the proof script for the preceding proposition.
1021	rewrite /dhyperball_polyveck /dsigning_bounded_polyveck	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1022	/hyperball_polyveck_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1023	apply dlist_point_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1024	+ by smt(mode_k_gt0).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1025	+ by apply hyperball_poly_point_bound_nonnegative.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1026	by move=> p; apply hyperball_poly_point_boundE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1027	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1028	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1029	lemma bounded_unit_signing_sample_pair_lossless md :	States checked lemma bounded_unit_signing_sample_pair_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
1030	is_lossless (dbounded_unit_signing_sample_pair md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1031	proof.	Begins the proof script for the preceding proposition.
1032	rewrite /dbounded_unit_signing_sample_pair.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1033	apply dlet_ll.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1034	+ by apply signing_unit_polyvecl_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1035	move=> y1 _.	Introduces universally quantified variables or hypotheses into the proof context.
1036	by apply dmap_ll; apply signing_unit_polyveck_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1037	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1038	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1039	lemma bounded_unit_signing_sample_pair_support md s :	States checked lemma bounded_unit_signing_sample_pair_support. The following proof script must derive this proposition from prior definitions and lemmas.
1040	s \in dbounded_unit_signing_sample_pair md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1041	signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1042	proof.	Begins the proof script for the preceding proposition.
1043	rewrite /dbounded_unit_signing_sample_pair supp_dlet.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1044	move=> [y1 [y1_supp]].	Introduces universally quantified variables or hypotheses into the proof context.
1045	rewrite supp_dmap.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1046	move=> [y2 [y2_supp ->]].	Introduces universally quantified variables or hypotheses into the proof context.
1047	have y1_unit : polyvecl_unit md y1	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1048	by apply signing_unit_polyvecl_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1049	have y2_unit : polyveck_unit md y2	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1050	by apply signing_unit_polyveck_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1051	rewrite /signing_sample_pair_wf /signing_sample_pair_unit	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1052	/signing_sample_pair_y1 /signing_sample_pair_y2 /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
1053	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1054	<code>+ split.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1055	<code>+ by move: y1_unit; rewrite /polyvecl_unit => -[h _].</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1056	<code>by move: y2_unit; rewrite /polyveck_unit => -[h _].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1057	<code>by split.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1058	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1059	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1060	<code>lemma bounded_unit_signing_sample_pair_sample_ok md s :</code>	States checked lemma <code>bounded_unit_signing_sample_pair_sample_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1061	<code>s \in dbounded_unit_signing_sample_pair md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1062	<code>signing_sample_pair_sample_ok md s.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1063	<code>proof.</code>	Begins the proof script for the preceding proposition.
1064	<code>move=> hs.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1065	<code>move: (bounded_unit_signing_sample_pair_support md s hs)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1066	<code>=> [s_wf s_unit].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1067	<code>rewrite /signing_sample_pair_sample_ok.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1068	<code>split=> //.</code>	Splits a conjunction or structured goal into independent proof obligations.
1069	<code>by apply signing_sample_pair_unit_bounded.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1070	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1071	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1072	<code>lemma bounded_unit_signing_sample_pair_distribution_ok md :</code>	States checked lemma <code>bounded_unit_signing_sample_pair_distribution_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1073	<code>signing_sample_pair_distribution_ok md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1074	<code>(dbounded_unit_signing_sample_pair md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1075	<code>proof.</code>	Begins the proof script for the preceding proposition.
1076	<code>rewrite /signing_sample_pair_distribution_ok.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1077	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
1078	<code>+ by apply bounded_unit_signing_sample_pair_lossless.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1079	<code>by move=> s; apply bounded_unit_signing_sample_pair_sample_ok.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1080	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1081	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1082	<code>lemma checked_hyperball_signing_sample_pair_lossless md :</code>	States checked lemma <code>checked_hyperball_signing_sample_pair_lossless</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1083	<code>is_lossless (dchecked_hyperball_signing_sample_pair md).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1084	<code>proof.</code>	Begins the proof script for the preceding proposition.
1085	<code>rewrite /dchecked_hyperball_signing_sample_pair.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1086	<code>apply dlet_l1.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1087	<code>+ by apply signing_bounded_polyvecl_lossless.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1088	<code>move=> y1 _.</code>	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
1089	by apply dmap_ll; apply signing_bounded_polyveck_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1090	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1091	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1092	lemma checked_hyperball_signing_sample_pair_support md s :	States checked lemma <code>checked_hyperball_signing_sample_pair_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1093	s \in dchecked_hyperball_signing_sample_pair md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1094	signing_sample_pair_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1095	proof.	Begins the proof script for the preceding proposition.
1096	rewrite /dchecked_hyperball_signing_sample_pair supp_dlet.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1097	move=> [y1 [y1_supp]].	Introduces universally quantified variables or hypotheses into the proof context.
1098	rewrite supp_dmap.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1099	move=> [y2 [y2_supp ->]].	Introduces universally quantified variables or hypotheses into the proof context.
1100	have y1_bounded : polyvecl_bounded_by md (signing_sample_bound md) y1	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1101	by apply signing_bounded_polyvecl_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1102	have y2_bounded : polyveck_bounded_by md (signing_sample_bound md) y2	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1103	by apply signing_bounded_polyveck_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1104	rewrite /signing_sample_pair_sample_ok /signing_sample_pair_wf	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1105	/signing_sample_pair_bounded	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1106	/signing_sample_pair_y1 /signing_sample_pair_y2 /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1107	split.	Splits a conjunction or structured goal into independent proof obligations.
1108	+ split.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1109	+ by move: y1_bounded; rewrite /polyvecl_bounded_by => -[h _].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1110	by move: y2_bounded; rewrite /polyveck_bounded_by => -[h _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1111	by split.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1112	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1113	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1114	lemma checked_hyperball_signing_sample_pair_distribution_ok md :	States checked lemma
1115	signing_sample_pair_distribution_ok md	<code>checked_hyperball_signing_sample_pair_distribution_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1116	(dchecked_hyperball_signing_sample_pair md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1117	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1118	rewrite /signing_sample_pair_distribution_ok.	Begins the proof script for the preceding proposition.
1119	split.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1120	+ by apply checked_hyperball_signing_sample_pair_lossless.	Splits a conjunction or structured goal into independent proof obligations.
1121	by move=> s; apply checked_hyperball_signing_sample_pair_support.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1122	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1123	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1124	lemma hyperball_sample_ok_sample_ok md s :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
		States checked lemma <code>hyperball_sample_ok_sample_ok</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1125	hyperball_sample_ok md s =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1126	signing_sample_pair_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1127	proof.	Begins the proof script for the preceding proposition.
1128	rewrite /hyperball_sample_ok /signing_sample_pair_sample_ok	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1129	/signing_sample_pair_wf /signing_sample_pair_bounded	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1130	/hyperball_polyvecl_ok /hyperball_polyveck_ok.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1131	move=> [y1_ok y2_ok].	Introduces universally quantified variables or hypotheses into the proof context.
1132	split.	Splits a conjunction or structured goal into independent proof obligations.
1133	+ split.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1134	+ by move: y1_ok; rewrite /polyvecl_bounded_by => -[h _].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1135	by move: y2_ok; rewrite /polyveck_bounded_by => -[h _].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1136	by split.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1137	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1138	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1139	lemma exact_hyperball_signing_sample_pair_lossless md :	States checked lemma exact_hyperball_signing_sample_pair_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
1140	is_lossless (dexact_hyperball_signing_sample_pair md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1141	proof.	Begins the proof script for the preceding proposition.
1142	rewrite /dexact_hyperball_signing_sample_pair.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1143	rewrite dprod_ll.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1144	split.	Splits a conjunction or structured goal into independent proof obligations.
1145	+ by apply hyperball_polyvecl_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1146	by apply hyperball_polyveck_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1147	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1148	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1149	lemma exact_hyperball_signing_sample_pair_support md s :	States checked lemma exact_hyperball_signing_sample_pair_support. The following proof script must derive this proposition from prior definitions and lemmas.
1150	s \in dexact_hyperball_signing_sample_pair md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1151	hyperball_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1152	proof.	Begins the proof script for the preceding proposition.
1153	case: s => y1 y2.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1154	rewrite /dexact_hyperball_signing_sample_pair supp_dprod /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1155	move=> [y1_supp y2_supp].	Introduces universally quantified variables or hypotheses into the proof context.
1156	rewrite /hyperball_sample_ok /signing_sample_pair_y1	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1157	/signing_sample_pair_y2 /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1158	split.	Splits a conjunction or structured goal into independent proof obligations.
1159	+ by apply hyperball_polyvecl_support.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1160	by apply hyperball_polyveck_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1161	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1162	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1163	lemma exact_hyperball_signing_sample_pair_supportP md s :	States checked lemma exact_hyperball_signing_sample_pair_supportP. The following proof script must derive this proposition from prior definitions and lemmas.
1164	s \in dexact_hyperball_signing_sample_pair md <=>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1165	hyperball_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1166	proof.	Begins the proof script for the preceding proposition.
1167	case: s => y1 y2.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1168	rewrite /dexact_hyperball_signing_sample_pair supp_dprod /hyperball_sample_ok	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1169	/signing_sample_pair_y1 /signing_sample_pair_y2 /.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1170	by rewrite hyperball_polyvecl_supportP hyperball_polyveck_supportP.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1171	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1172	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1173	lemma exact_hyperball_signing_sample_pair_sample_ok md s :	States checked lemma exact_hyperball_signing_sample_pair_sample_ok. The following proof script must derive this proposition from prior definitions and lemmas.
1174	s \in dexact_hyperball_signing_sample_pair md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1175	signing_sample_pair_sample_ok md s.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1176	proof.	Begins the proof script for the preceding proposition.
1177	move=> hs.	Introduces universally quantified variables or hypotheses into the proof context.
1178	apply hyperball_sample_ok_sample_ok.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1179	by apply exact_hyperball_signing_sample_pair_support.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1180	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1181	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1182	lemma exact_hyperball_signing_sample_pair_distribution_ok md :	States checked lemma exact_hyperball_signing_sample_pair_distribution_ok. The following proof script must derive this proposition from prior definitions and lemmas.
1183	signing_sample_pair_distribution_ok md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1184	(dexact_hyperball_signing_sample_pair md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1185	proof.	Begins the proof script for the preceding proposition.
1186	rewrite /signing_sample_pair_distribution_ok.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1187	split.	Splits a conjunction or structured goal into independent proof obligations.
1188	+ by apply exact_hyperball_signing_sample_pair_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1189	by move=> s; apply exact_hyperball_signing_sample_pair_sample_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1190	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1191	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1192	lemma exact_hyperball_signing_sample_pair_point_bound md s :	States checked lemma exact_hyperball_signing_sample_pair_point_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1193	mu (dexact_hyperball_signing_sample_pair md) (pred1 s) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1194	hyperball_sample_pair_point_bound md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1195	proof.	Begins the proof script for the preceding proposition.
1196	case: s => y1 y2.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.

Line	EasyCrypt source	Mathematical reading
1197	rewrite /dexact_hyperball_signing_sample_pair	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1198	/hyperball_sample_pair_point_bound dprod1E /=. .	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1199	apply ler_pmul.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1200	+ by apply ge0_mu.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1201	+ by apply ge0_mu.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1202	+ by apply hyperball_polyvec1_point_boundE.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1203	by apply hyperball_polyveck_point_boundE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1204	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1205	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1206	lemma exact_hyperball_signing_sample_pair_checkedE md :	States checked lemma exact_hyperball_signing_sample_pair_checkedE. The following proof script must derive this proposition from prior definitions and lemmas.
1207	dexact_hyperball_signing_sample_pair md =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1208	dchecked_hyperball_signing_sample_pair md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1209	proof.	Begins the proof script for the preceding proposition.
1210	rewrite /dexact_hyperball_signing_sample_pair	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1211	/dchecked_hyperball_signing_sample_pair	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1212	/dhyperball_polyvec1 /dhyperball_polyveck.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1213	rewrite dprod_dlet.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1214	apply eq_dlet => // y1.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1215	by rewrite /dmap /(\o).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1216	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1217	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1218	end HAETAE_Distributions.	Closes the current theory or module block.